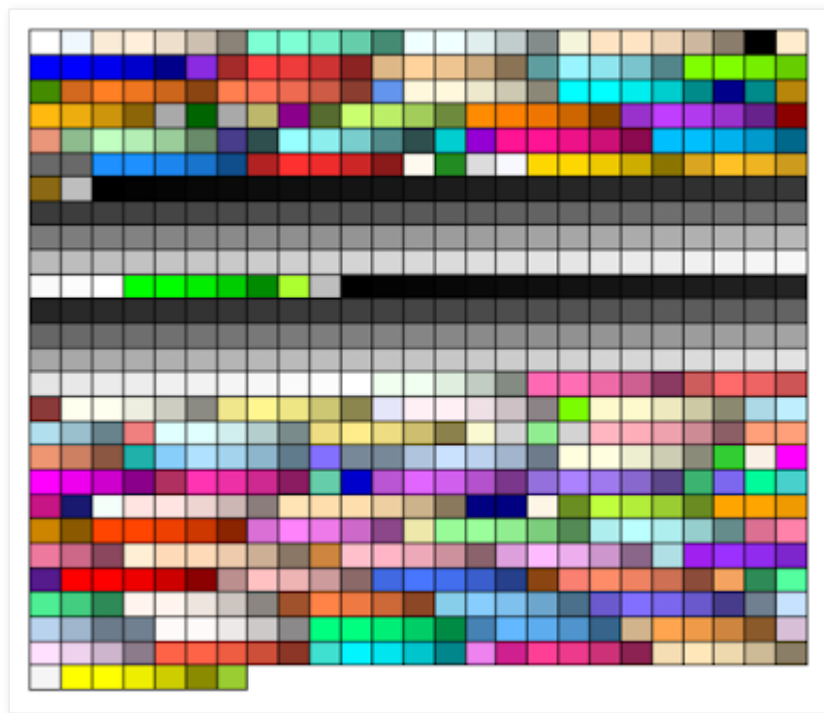


Nomi e codici dei colori di R

Nella installazione base di **R** sono disponibili i 657 colori qui rappresentati



i cui nomi vengono visualizzati nella Console di R digitando:

colors()

Tuttavia per visualizzare di ogni colore contemporaneamente:

→ il nome da impiegare per configurare il colore nelle funzioni che lo prevedono

→ un tassello colorato che riporta il colore corrispondente

→ il codice esadecimale che può essere impiegato in alternativa al nome

è necessario disporre di una tabella di corrispondenza che deve essere generata con uno script (non banale) come quello qui riportato, che è stato adattato da [1].

Innanzitutto se non l'avete già fatto create la cartella C:\Rdati

Lo script salva **la tabella dei colori di R** nel file C:\Rdati\Rcolors.pdf – tenete il file a portata di mano, vi servirà quando vorrete impiegare i colori di **R**.

Copiate lo script, incollatelo nella Console di R e premete ↵ Invio.

```
# LA TABELLA DEI COLORI DI R
# Tratto e adattato da: Earl F Glynn. Color Chart in R.
# GitHub, https://bit.ly/2WJrJrR
#
# Lo script salva la tabella dei colori di R nel file C:\Rdati\Rcolors.pdf
#
pdf("C:/Rdati/Rcolors.pdf", width=6, height=10)
oldparameters <- par(mar=c(1,1,2,1), mfrow=c(2,1))
#
stopifnot(length(colors()) == 657)
SetTextContrastColor <- function(color)
{ ifelse( mean(col2rgb(color)) > 127, "black", "white") }
```

```

TextContrastColor <- unlist( lapply(colors(), SetTextContrastColor) )
#
# tabella dei colori di R ordinati per numero
#
colCount <- 25 # number per row
rowCount <- 27
plot( c(1,colCount), c(0,rowCount), type="n", ylab="", xlab="", axes=FALSE,
ylim=c(rowCount,0))
title("Tabella dei colori di R")
for (j in 0:(rowCount-1))
{ base <- j*colCount
remaining <- length(colors()) - base
RowSize <- ifelse(remaining < colCount, remaining, colCount)
rect((1:RowSize)-0.5,j-0.5, (1:RowSize)+0.5,j+0.5, border="black", col=colors()[base +
(1:RowSize)])
text((1:RowSize), j, paste(base + (1:RowSize)), cex=0.7, col=TextContrastColor[base +
(1:RowSize)])
}
#
# colori di R ordinati per tonalità e saturazione
#
RGBColors <- col2rgb(colors()[1:length(colors())])
HSVColors <- rgb2hsv( RGBColors[1,], RGBColors[2,], RGBColors[3,],
maxColorValue=255)
HueOrder <- order( HSVColors[1,], HSVColors[2,], HSVColors[3,] )
plot(0, type="n", ylab="", xlab="", axes=FALSE, ylim=c(rowCount,0),
xlim=c(1,colCount))
title("Colori di R ordinati per tonalità e saturazione")
for (j in 0:(rowCount-1))
{for (i in 1:colCount)
{k <- j*colCount + i
if (k <= length(colors()))
{rect(i-0.5,j-0.5, i+0.5,j+0.5, border="black", col=colors()[ HueOrder[k] ])
text(i,j, paste(HueOrder[k]), cex=0.7, col=TextContrastColor[ HueOrder[k] ])}}}
#
# colori di R con nomi e codice esadecimale del colore
#
GetColorHexAndDecimal <- function(color)
{c <- col2rgb(color)
sprintf("#%02X%02X%02X %3d %3d %3d", c[1],c[2],c[3], c[1], c[2], c[3])}
par(oldparameters)
oldparameters <- par(mar=c(1,1,1,1))
index <- paste(1:length(colors()))
HexAndDec <- unlist( lapply(colors(), GetColorHexAndDecimal) )
PerColumn <- 50
PerPage <- 2*PerColumn
for (page in 1: (trunc( (length(colors()) + (PerPage-1)) / PerPage) ) )
{plot(0, type="n", ylab="", xlab="",
axes=FALSE, ylim=c(PerColumn,0), xlim=c(0,1))
title("Colori di R con nome e codice esadecimale")
mtext(paste("pag.", page, "/ 7"), SOUTH<-1, adj=1, line=-1)
base <- PerPage*(page-1)
remaining <- length(colors()) - base
ColumnSize <- ifelse(remaining < PerColumn, remaining, PerColumn)
rect(0.00, 0:(ColumnSize-1), 0.49, 1:ColumnSize, border="black", col=colors()[(base+1):
(base+ColumnSize)])

```

```

text(0.045, 0.45+(0:(ColumnSize-1)), adj=1, index[(base+1):(base+ColumnSize)],
cex=0.6, col=TextContrastColor[(base+1):(base+ColumnSize)])
text(0.06, 0.45+(0:(ColumnSize-1)), adj=0, colors()[(base+1):(base+ColumnSize)],
cex=0.6, col=TextContrastColor[(base+1):(base+ColumnSize)])
save <- par(family="mono") # use mono-spaced font with number columns
text(0.25, 0.45+(0:(ColumnSize-1)), adj=0, HexAndDec[(base+1):(base+ColumnSize)],
cex=0.6, col=TextContrastColor[(base+1):(base+ColumnSize)])
par(save)
if (remaining > PerColumn)
{remaining <- remaining - PerColumn
ColumnSize <- ifelse(remaining < PerColumn, remaining, PerColumn)
rect(0.51, 0:(ColumnSize-1), 1.00, 1:ColumnSize, border="black", col=colors()
[(base+PerColumn+1):(base+PerColumn+ColumnSize)])
text(0.545, 0.45+(0:(ColumnSize-1)), adj=1, index[(base+PerColumn+1):
(base+PerColumn+ColumnSize)], cex=0.6, col=TextContrastColor[(base+PerColumn+1):
(base+PerColumn+ColumnSize)])
text(0.56, 0.45+(0:(ColumnSize-1)), adj=0, colors()[(base+PerColumn+1):
(base+PerColumn+ColumnSize)], cex=0.6, col=TextContrastColor[(base+PerColumn+1):
(base+PerColumn+ColumnSize)])
save <- par(family="mono")
text(0.75, 0.45+(0:(ColumnSize-1)), adj=0, HexAndDec[(base+PerColumn+1):
(base+PerColumn+ColumnSize)], cex=0.6, col=TextContrastColor[(base+PerColumn+1):
(base+PerColumn+ColumnSize)])
par(save)}}
par(oldparameters)
dev.off()
#

```

Se volete fare un passo ulteriore, cioè in alternativa a quelli qui riportati generare vostri nuovi **colori personalizzati** e raccogliarli poi in vostre **palette** – le tavolozze che potete poi impiegare per applicare automaticamente colori differenti alle diverse componenti di un grafico – potete trovare le indicazioni nei post:

- [Come realizzare colori personalizzati \[1\]](#)
- [Come realizzare colori personalizzati \[2\]](#)

[1] Earl F. Glynn. *Color Chart in R*. Pubblicato su GitHub.
<https://github.com/EarlGlynn/colorchart/wiki/Color-Chart-in-R>
